Hide & Seek

They hide. I seek.

Status: work in progress. The first Tkinter desktop interface is running, and I am still reviewing the detection logic, the validation and the explanations shown to the analyst. This is a learning project, not a production security product.

Hide & Seek is a Python lab project that looks for potentially suspicious Business Email Compromise (BEC) activity in Microsoft 365 audit logs. It focuses on mailbox forwarding and inbox rules, the things an attacker can set up after taking over an account to keep reading mail or to hide important messages from the owner.

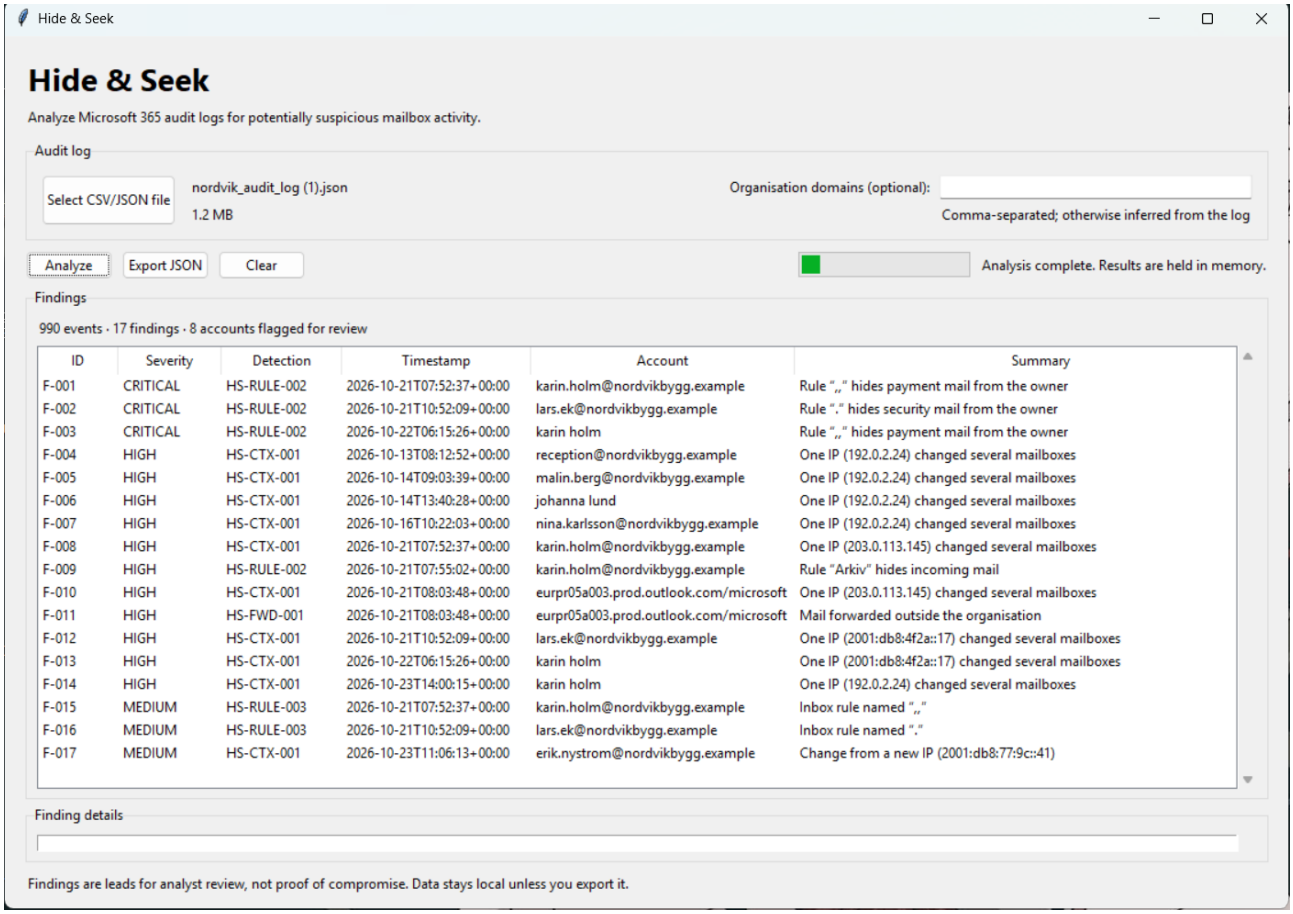
I am moving the main workflow from a browser-based Streamlit dashboard to a local Tkinter desktop application. The goal is to practise handling sensitive audit data responsibly, explain findings clearly, and learn how an analyst tells a suspicious indicator apart from a confirmed incident. The original Streamlit interface is kept for comparison while I make the switch.

Screenshots

All screenshots use a synthetic audit log for a fictional company (nordvikbygg.example). Addresses and IPs are made up and the IPs come from documentation ranges.

Main window

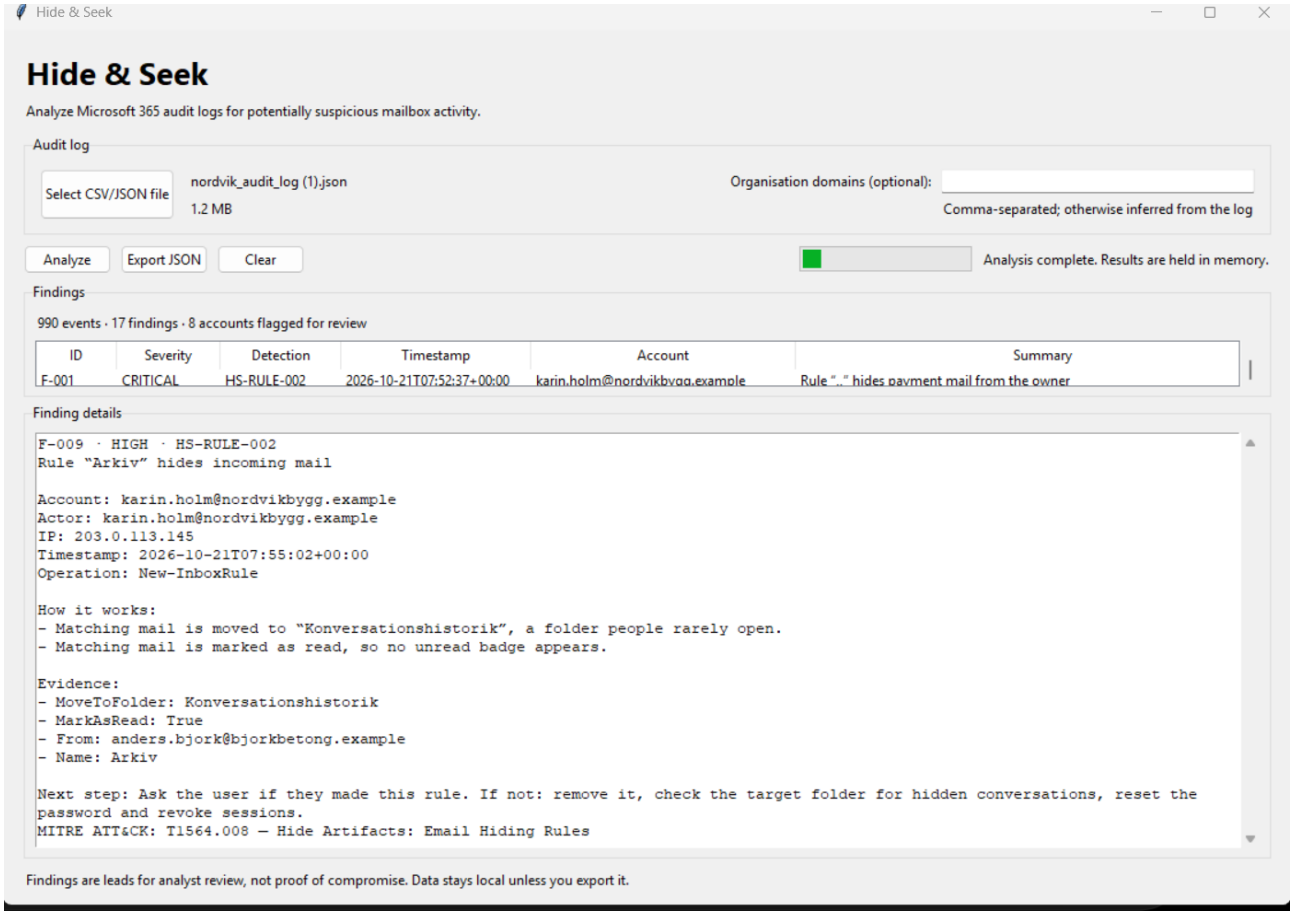
The window after analysing a synthetic log with 990 events. It shows 17 findings and 8 accounts flagged for review.



Main window with the findings list

Finding details

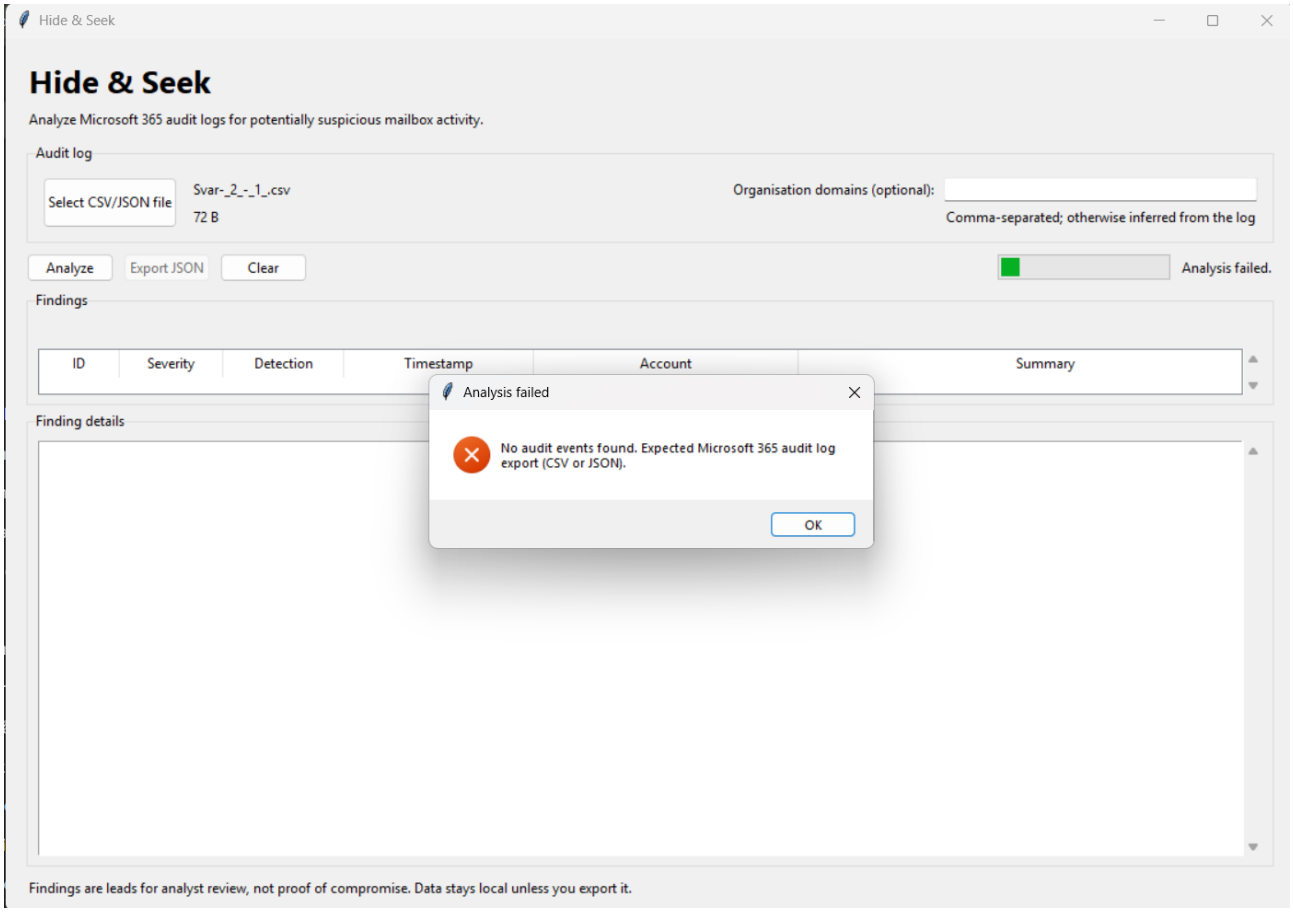
Selecting a finding shows which rule fired, how the hidden rule works, the raw evidence, a suggested next step and the MITRE ATT&CK mapping.



Details for finding F-009

Input validation

A file that does not contain any audit events is rejected with a clear message instead of a traceback. Here a 72 byte CSV was selected and analysed.



Analysis failed dialog

The problem

After phishing a password, a BEC attacker rarely does anything loud. They may:

1. Forward mail to an external address, so they keep reading it even after a password reset.
2. Create an inbox rule that moves invoice replies or security warnings into a folder nobody opens.
3. Mark messages as read or delete them, so the mailbox owner is less likely to notice.

Then they can reply in a real invoice thread with new bank details while the victim never sees the answers. These changes show up in the audit log, but they are easy to miss among thousands of normal events. Hide & Seek pulls them out and explains in plain language what each one does.

What it looks for

Severity is assigned by the current detection logic. I am still checking whether the reasoning behind each severity is consistent and easy to understand.

Rule	Looks for	Severity	MITRE ATT&CK
HS-FWD-001	Set-Mailbox forwarding (ForwardingSmtpAddress, ForwardingAddress). Higher severity when forwarding is external and no copy is kept	Low to Critical	T1114.003
HS-RULE-001	Inbox rules using ForwardTo, RedirectTo or ForwardAsAttachmentTo, including possible lookalike domains	Medium to High	T1114.003

Rule	Looks for	Severity	MITRE ATT&CK
HS-RULE-002	Inbox rules that hide mail by moving it to certain folders, marking it as read or deleting it. Payment and security keywords (English and Swedish) raise the severity	Medium to Critical	T1564.008
HS-RULE-003	Rule names such as . or .. that are easy to overlook	Medium	T1564.008
HS-CTX-001	Context signals: a first-seen IP, one IP changing several mailboxes, or an IP linked to failed sign-ins	Medium to High	T1078

Correlation: the engine raises severity when a mailbox has both forwarding and a message-hiding rule, because that combination fits a typical BEC pattern.

These are indicators for review, not proof that an account is compromised. Severity is not the same thing as confidence, and neither one should be treated as a confirmed incident on its own.

Current functionality

The Tkinter app currently lets you:

- Select a CSV or JSON audit log from your computer.
- Optionally enter your organisation's domains (otherwise they are inferred from the log).
- Analyse the file with the Hide & Seek detection engine.
- Review the findings in a table and open the details of a selected finding.
- Export the report as JSON.
- Clear the current analysis from the window.

The detection engine has no UI code. The function analyze_bytes() returns a plain Python dictionary, so the same engine is used by the Tkinter app, the original Streamlit dashboard and the command-line interface.

Why Tkinter and a local-first workflow

Audit logs can contain personal data, account names, IP addresses and information about what users have been doing. For this project I want the default workflow to be local and easy to inspect.

- The Tkinter app runs as a normal desktop program and does not start a web server.
- The selected file is analysed on the local computer. No cloud service or external API is used.
- Reports are never saved automatically. Exporting is something you choose to do.
- The Streamlit app is still available, but the desktop app is the main interface.

A desktop app removes the network exposure of a web application, but it does not make the program secure by itself. Malicious input files, parser bugs, mistakes in the detection logic, vulnerable dependencies and careless handling of exported reports are still risks. That is why the project needs testing and review as well as a local interface.

Run it on Windows

Tkinter is included with most standard Windows Python installations.

```
cd hide-and-seek
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python desktop_app.py
```

Select a CSV or JSON audit log, click Analyze, select a finding to see its details, and use Export JSON only when you want to save a report.

If PowerShell blocks the activation script because of your local policy, run .venv\Scripts\python.exe directly instead of changing the policy for the whole system.

Original Streamlit interface

The browser interface is kept while I compare the two frontends:

```
python -m streamlit run app.py
```

Streamlit starts a local web server, then open http://localhost:8501 and click Try with sample data. Do not deploy it publicly with real audit logs. For a portfolio demo, use synthetic data only.

Command line

The same engine can be run from the terminal:

```
python -m hideandseek sample_data/sample_audit_log.csv
python -m hideandseek my_log.json --org-domain contoso.com --json > report.json
```

The exit code is 1 when anything high or critical is found, so the CLI can be used in scripts. Check the CLI help for the options that are actually supported, and be careful not to redirect reports into a shared or tracked folder by accident.

Getting a real audit log

Only analyse logs from an organisation if you have explicit permission and follow its data-handling rules.

Purview portal: go to Audit, then Search. Choose the activities New-InboxRule, Set-InboxRule, Set-Mailbox and User signed in, then export the results as CSV.

PowerShell (Exchange Online Management module):

```
Connect-ExchangeOnline
$ops = "New-InboxRule","Set-InboxRule","Set-Mailbox","UserLoggedIn","UserLoginFailed"
Search-UnifiedAuditLog -StartDate (Get-Date).AddDays(-30) -EndDate (Get-Date) -Operations $ops -ResultSize 5000
  ForEach-Object { $_.AuditData | ConvertFrom-Json } |
  ConvertTo-Json -Depth 6 | Out-File audit.json -Encoding utf8
```

Also accepted: JSON Lines, JSON wrapped in {"value": [...]}, and simple flat CSV files with columns like Timestamp, Operation, User and ClientIP plus one column per parameter.

Sample data and safe use

The sample_data folder contains synthetic demonstration data only. The main sample is a made-up law firm (lexpartners.example) with 21 events: a normal week, a newsletter folder rule that is not flagged, a leaver's mail forwarded to a manager (low), a failed password spray, and an attacker who takes over two mailboxes. Documentation-range IPs and reserved .example domains are used throughout. You can regenerate the samples with python sample_data/make_samples.py.

For screenshots and tests, use synthetic data only. Do not commit real Microsoft 365 audit exports, mailbox contents, credentials, tokens, personal data or investigation reports to GitHub.

Security and privacy

- Local processing: the desktop workflow reads the selected file locally. There are no uploads, no telemetry and no external API requests.

- Input validation: only CSV and JSON are supported. Files are limited to 10 MB, and parsing also limits the number of records, field lengths and JSON nesting depth. These limits need to stay covered by tests as the code changes.
- Untrusted values: everything in the log is treated as data. The project does not use eval(), pickle, shell commands or any other unsafe handling of log content.
- Limited persistence: the file and the report stay in memory until you export a report or clear the window. Clearing does not guarantee secure erasure from memory.
- Controlled export: reports are exported as JSON only on request, with an overwrite confirmation. Exported files can still contain sensitive information and should be stored with care. CSV export is left out on purpose to avoid spreadsheet formula injection.
- No automatic remediation: the tool never changes mailbox settings or acts in a Microsoft 365 tenant.
- Frontend differences: the Streamlit frontend escapes every log value as HTML before showing it. That protection does not carry over to Tkinter, so each frontend has to be reviewed for how it displays untrusted data.

Tests

```
python -m pytest
```

The test suite covers the detection rules, false-positive scenarios, event correlation, IP context, file parsing, input limits, malformed data, and the desktop file handling and report workflow. The number of tests changes as I work, so run the suite yourself and look at the actual result.

What I am improving next

1. Make severity easier to explain. Trace how each severity is assigned, write down the evidence behind it and look for inconsistent or too strong labels. Keep potential impact, strength of evidence and confirmed incident status separate.
2. Give the analyst more context. Make it clear which rule fired, what triggered it, why it matters and what to check next.
3. Add UpdateInboxRules. Rules created in Outlook can appear under this operation with less detail in the audit record.
4. Keep testing defensive handling with malformed, unexpected and oversized files and unusual field values, without weakening the existing regression tests.
5. Check that the desktop app and the engine agree. The Tkinter window should show the same findings as the engine and never change severity or evidence.
6. Look at how accounts are shown. In the screenshot some findings show a display name (karin holm) or a server name (eurpr05a003.prod.outlook.com/microsoft) in the Account column instead of a mailbox address. I want to check whether that comes from the log format or from the parser.
7. Improve wording and documentation. Prefer "accounts flagged for review" over "compromised accounts" unless compromise has been confirmed another way.

Later I may package the desktop app so it is easier to install, but first I want to understand and verify the basics.

Limitations

- This is a triage and learning tool. A finding is a lead to follow up, not proof of compromise, and the tool does not replace Microsoft security tooling or an organisation's incident-response process.
- New IP findings depend on the history in the log. Export at least a couple of weeks so there is a normal baseline.
- UpdateInboxRules coverage is planned but not done yet.
- There is no IP geolocation or threat-intelligence enrichment.
- A clean result does not prove that a mailbox is safe. Missing events, incomplete history, unsupported formats or rules the tool does not recognise can all affect the result.
- Input limits reduce some resource-exhaustion risks, but the app is not a full parser sandbox or a security certification.

Project structure

```
hide-and-seek/
|-- app.py                    original Streamlit dashboard
|-- desktop_app.py            Tkinter desktop application
|-- frontend/                 original Streamlit frontend
|   |-- components.py         HTML building blocks (everything escaped)
|   `-- styles.css            styling for the Streamlit app
|-- hideandseek/              detection engine, no UI code
|   |-- parsers.py            CSV/JSON to normalised Event objects
|   |-- context.py            log-wide context such as IP history
|   |-- engine.py             rules, correlation and report building
|   |-- models.py             Event, Finding and Severity models
|   |-- utils.py              IP/domain helpers and keyword lists
|   |-- __main__.py           command-line interface
|   `-- rules/                one file per detection rule
|-- sample_data/              fictional audit logs and generator
|-- static/fonts/             bundled Montserrat (SIL OFL)
|-- tests/                    automated tests
`-- docs/
    |-- ADDING_RULES.md       how to write a new rule
    `-- screenshots/          screenshots used in this README
```

The repository may change while the desktop transition is in progress.

Adding a rule

Create one file in hideandseek/rules/, subclass Rule and add the @register decorator. It is picked up automatically. See docs/ADDING_RULES.md for a full example.

References

- MITRE ATT&CK: Email Forwarding Rule (T1114.003)
- MITRE ATT&CK: Hide Artifacts, Email Hiding Rules (T1564.008)
- MITRE ATT&CK: Valid Accounts (T1078)

This project is built for learning and portfolio demonstration. Feedback on the detection reasoning, the safe handling of audit data and the clarity of the analyst workflow is welcome.